

Máquinas de vectores de soporte

Desarrollo de Aplicaciones Avanzadas de Ciencias Computacionales
Inteligencia Artificial
(TC3002B.M2)

José Carlos Ortiz Bayliss



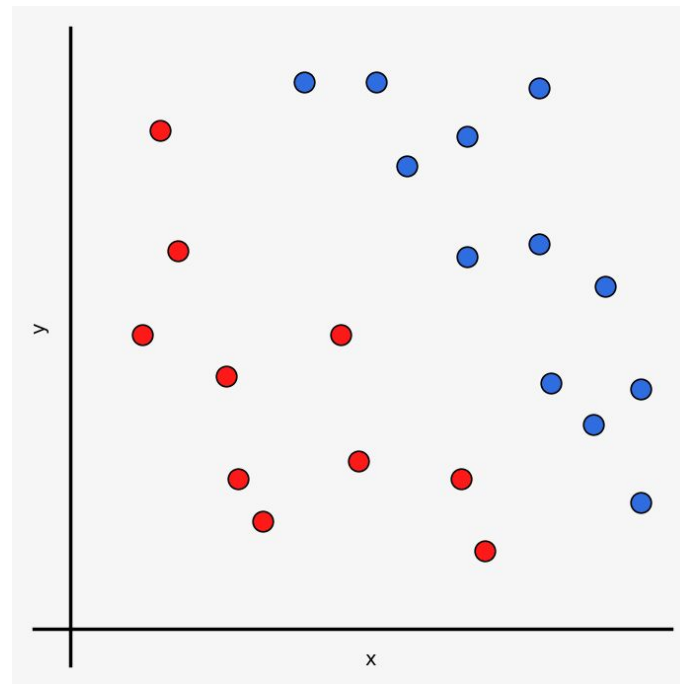
Las máquinas de vectores de soporte

- Las máquinas de vectores (SVM) de soporte son modelos de aprendizaje supervisado que se usan para regresión y clasificación.
- La idea principal es que SVM trata de separar los elementos pertenecientes a dos clases de la mejor manera posible, maximizando el espacio entre las dos clases (usando un hiperplano que separa ambas clases y transforma las características para tratar de mejorar la separación).
- El algoritmo SVM estándar está formulado para problemas de clasificación binaria (dos clases).

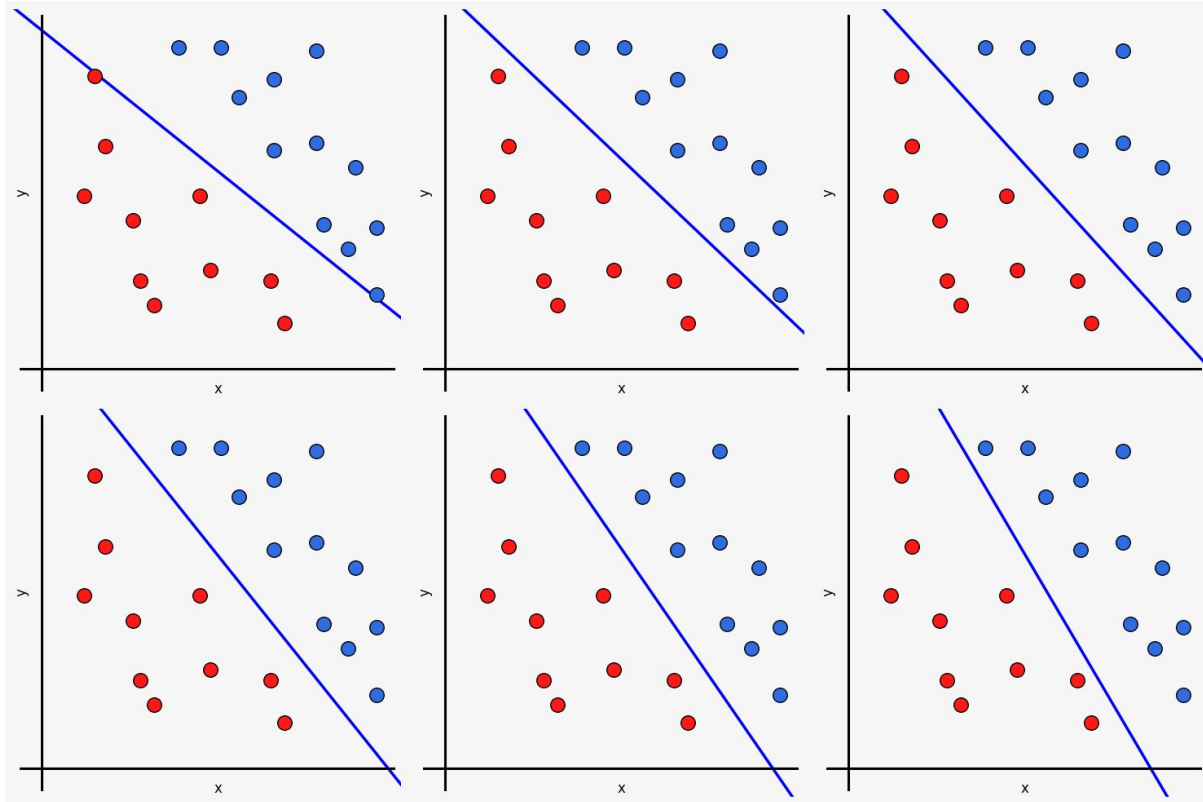
¿Cómo funciona una SVM?

- I. Toma como referencia el conjunto de puntos etiquetados por clase (originalmente diseñado para dos clases solamente).
- II. Busca rectas (o hiperplanos) que separen ambas clases. Existen muchas rectas posibles que separen perfectamente los datos.
- III. Evalúa el "margen" de cada recta. El margen es la distancia entre la recta y los puntos más cercanos de cada clase (espacio entre las clases).
- IV. Selecciona la recta con margen máximo. Dicha recta se convierte en la frontera (división entre ambas clases)*.

* La SVM permite que algunos puntos queden mal clasificados, pero penaliza esos errores mientras sigue intentando mantener el margen lo más grande posible.



Diferentes formas de separar linealmente ambas clases



Vectores de soporte

Los vectores de soporte son los puntos del conjunto de entrenamiento que determinan la frontera de decisión del modelo.

- Están más cerca de la frontera.
- Tocaban el margen.
- Violan el margen (en margen suave).

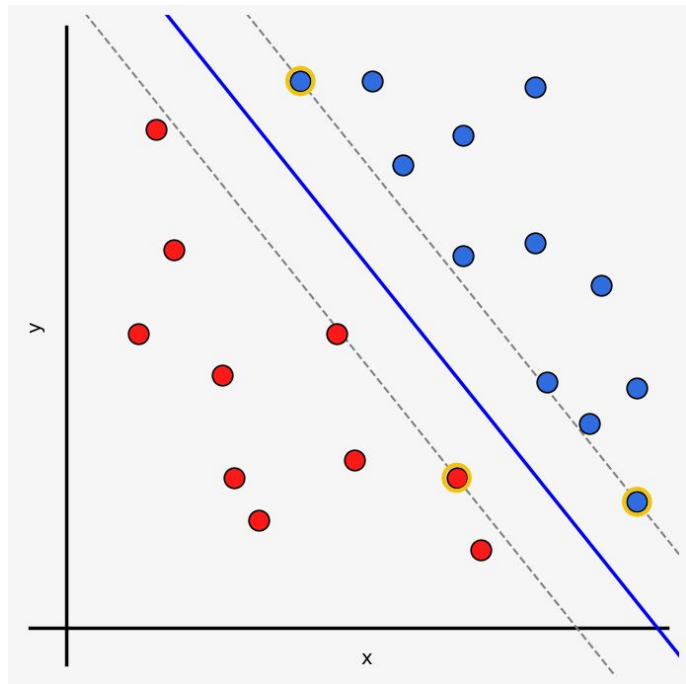
Los vectores de soporte son los únicos puntos que influyen en el modelo.

- Si se elimina uno, la frontera cambia.
- Si se elimina un punto que no corresponde a un vector de soporte, la frontera permanece igual.

Maximización del margen

Entre todas las rectas (o hiperplanos) que separan correctamente las clases, la SVM elige aquella que deja la mayor distancia posible entre la frontera y los puntos más cercanos de cada clase.

- Ese espacio se llama margen. Los puntos que "tocan" ese margen son los vectores de soporte.
- Maximizar el margen:
 - Hace la solución más robusta al ruido,
 - Reduce el riesgo de sobreajuste,
 - Mejora la capacidad de generalización.

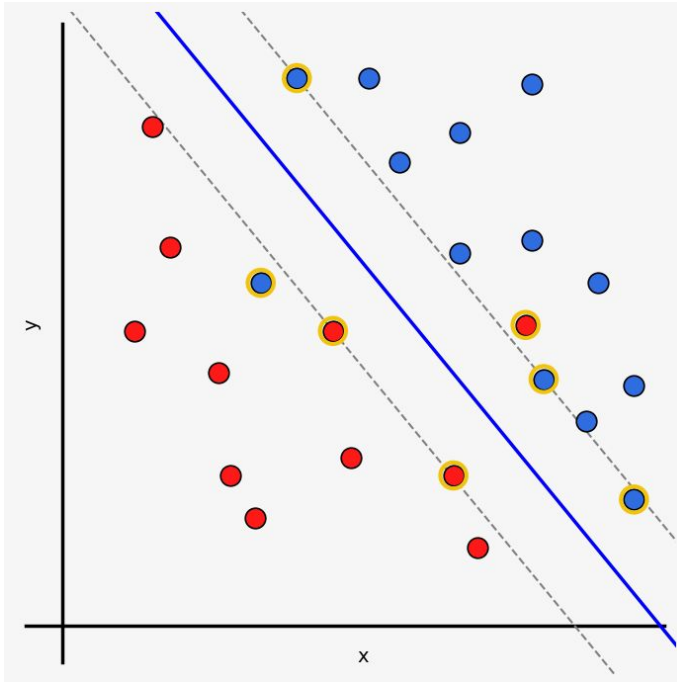


¿Qué sucede cuando las clases no son linealmente separables?

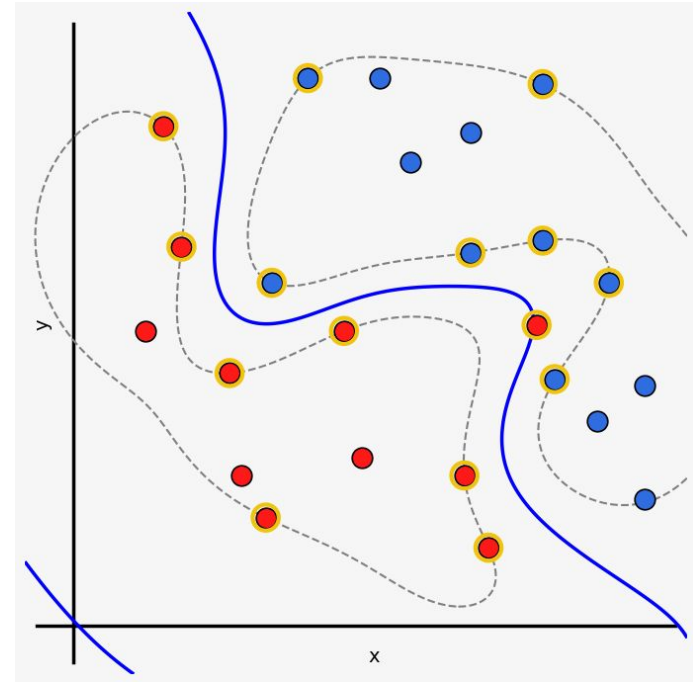
- Encontrar el máximo margen es mucho más fácil bajo el supuesto de que las clases son linealmente separables.
- En el caso de los datos no linealmente separables, debemos aceptar que algunos datos no serán correctamente clasificados. En estos casos, el margen debe ser "suave".
- No siempre necesitas aceptar errores si usas kernels. Un kernel puede hacer que datos no linealmente separables en el espacio original se vuelvan separables o más fácilmente separables en un espacio implícito, aunque no siempre elimina la necesidad de errores.

¿Qué sucede cuando las clases no son linealmente separables?

Margen suave



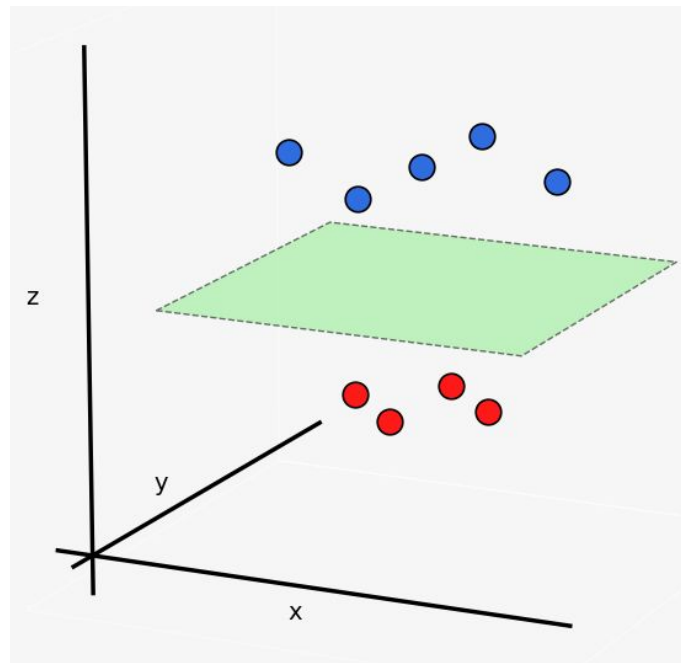
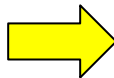
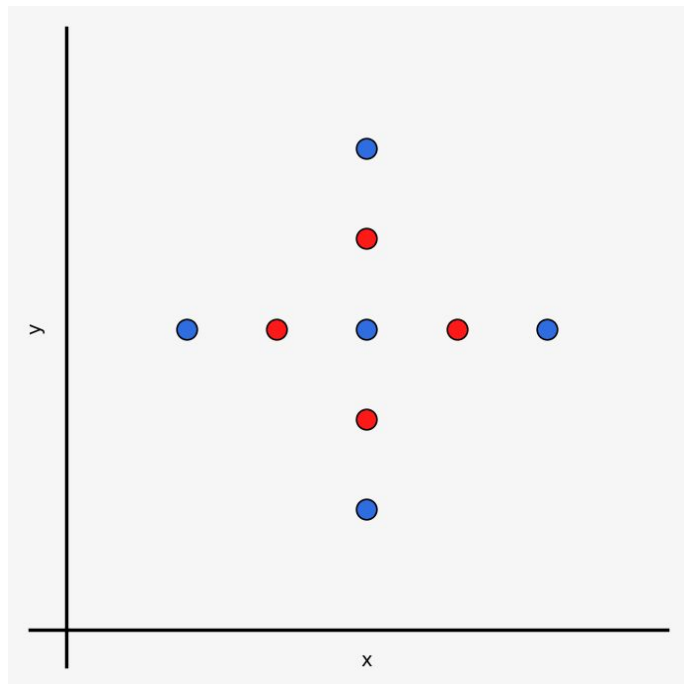
Kernel (RBF)



El kernel

- Un kernel permite trabajar los datos como si éstos hubieran sido transformados a un espacio de mayor dimensión (sin calcular explícitamente esa transformación).
- Un kernel permite considerar los datos en un espacio de mayor dimensión de forma implícita, facilitando encontrar separaciones más simples.
- Un kernel puede hacer que problemas no linealmente separables en el espacio original se vuelvan más fácilmente separables en un espacio transformado.

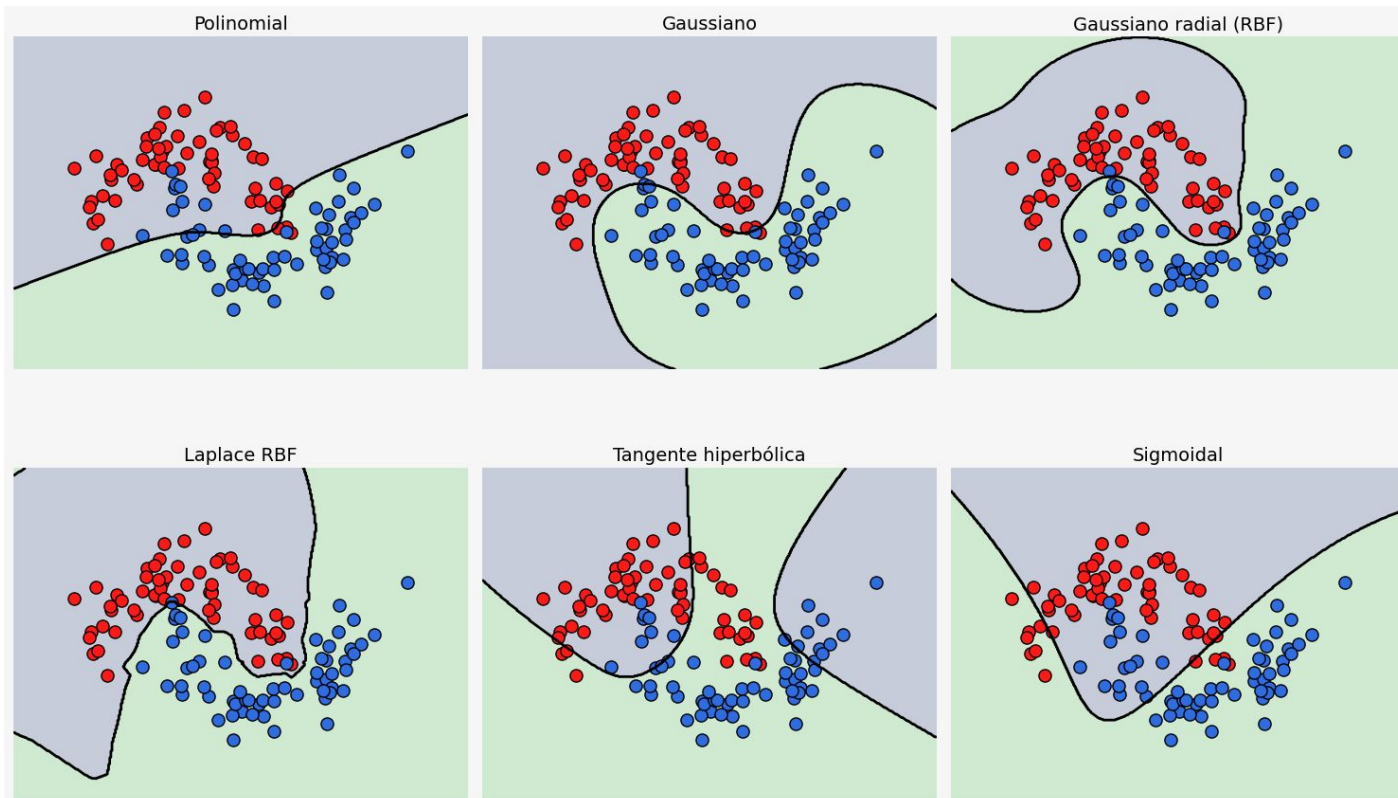
El efecto del kernel



Los kernels más populares

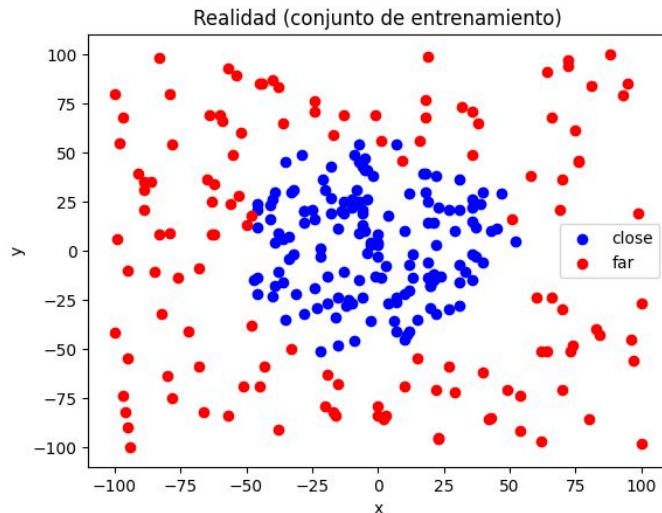
- **Polinomial.** Captura relaciones donde la interacción entre variables importa (combinaciones, curvaturas suaves). Permite fronteras más complejas que una línea recta, controlando el grado de la curva.
- **Gaussiano.** Asigna mayor similitud a puntos cercanos y la reduce rápidamente con la distancia. Genera superficies suaves y localmente adaptativas.
- **Gaussiano radial (RBF).** Es la versión más usada del gaussiano: crea "burbujas" de influencia alrededor de cada punto. Muy flexible para modelar patrones no lineales complejos sin necesidad de definir explícitamente la forma.
- **Laplace RBF.** Similar al RBF, pero con una caída de similitud más abrupta. Tiende a ser más sensible a cambios locales y puede adaptarse mejor a datos con variaciones más marcadas.
- **Tangente hiperbólica.** Imita el comportamiento de una neurona en redes neuronales. Puede capturar no linealidades tipo "S", aunque es menos estable que otros kernels y más sensible a la escala de los datos.
- **Sigmoidal.** Muy cercano al de tangente hiperbólica; modela transiciones suaves entre clases. Funciona bien en ciertos escenarios, pero no siempre garantiza buen desempeño y depende bastante de los parámetros.

Los kernels más populares



Clasificación usando máquinas de vectores de soporte

X	Y	Class
-38	-11	close
-86	35	far
32	73	far
21	-15	close
12	-7	close



Clasificación usando máquinas de vectores de soporte

```
# Importamos los módulos necesarios
import numpy as np
import pandas as pd
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

# Leemos los datos que usaremos
points = pd.read_csv("points.csv")

# Separamos las variables independientes de la dependiente
X = points.drop("Class", axis = 1).values
y = points["Class"].values

# Dividimos los datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.60, random_state = 54321)
```

Clasificación usando máquinas de vectores de soporte

```
# Creamos un modelo de clasificación usando SVM (con kernel lineal)
model = svm.LinearSVC()

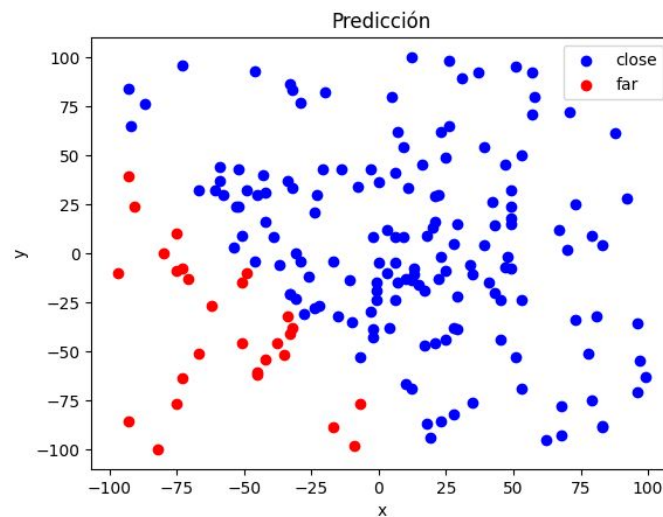
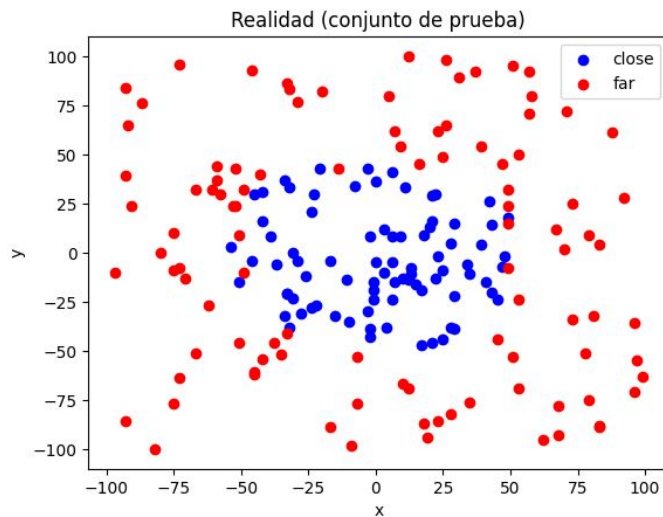
# Entrenamos el modelo usando los datos de entrenamiento
model.fit(X_train, y_train)

# Realizamos la predicción usando los datos de prueba
prediction = model.predict(X_test)

# Generamos la matriz de confusión
confusion_matrix(y_test, prediction)
```

Análisis del desempeño

		Predicción	
		close	far
Realidad	close	79	9
	far	66	35



Clasificación de puntos según su distancia (kernel no lineal)

```
# Creamos un modelo de clasificación usando SVM (con kernel RBF)
model = svm.SVC()

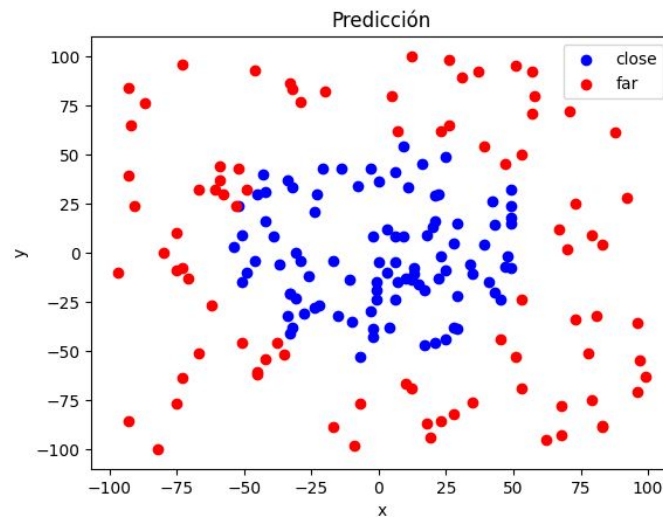
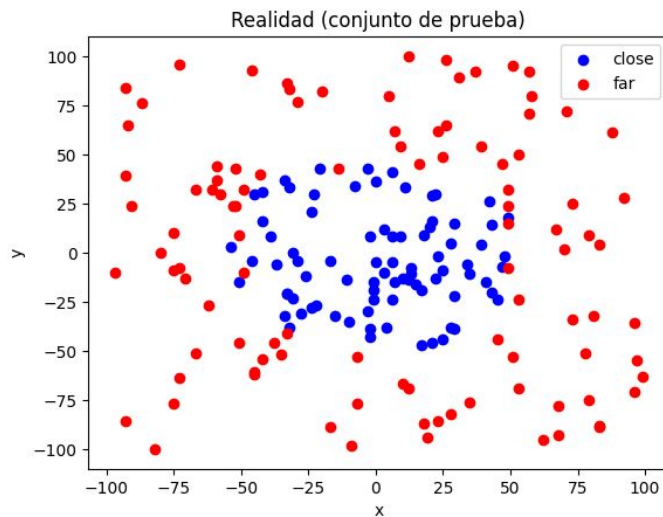
# Entrenamos el modelo usando los datos de entrenamiento
model.fit(X_train, y_train)

# Realizamos la predicción usando los datos de prueba
prediction = model.predict(X_test)

# Generamos la matriz de confusión
confusion_matrix(y_test, prediction)
```

Análisis del desempeño

		Predicción	
		close	far
Realidad	close	82	0
	far	14	87



Problemas generados por la escala de las variables

```
# Importamos los módulos necesarios
import numpy as np
import pandas as pd
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

# Leemos los datos que usaremos
points = pd.read_csv("distance.csv")

# Separamos las variables independientes de la dependiente
X = points.drop("Output", axis = 1).values
y = points["Output"].values

# Dividimos los datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.60, random_state = 54321)
```

Problemas generados por la escala de las variables

```
# Creamos un modelo de clasificación usando SVM (con kernel RBF)
model = svm.SVC()

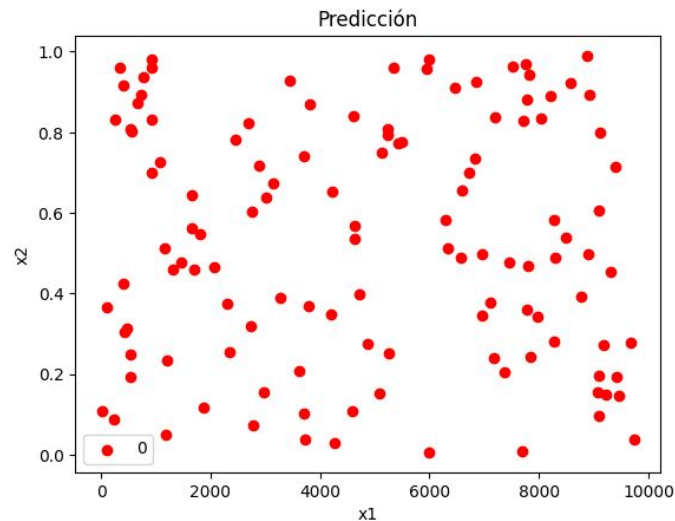
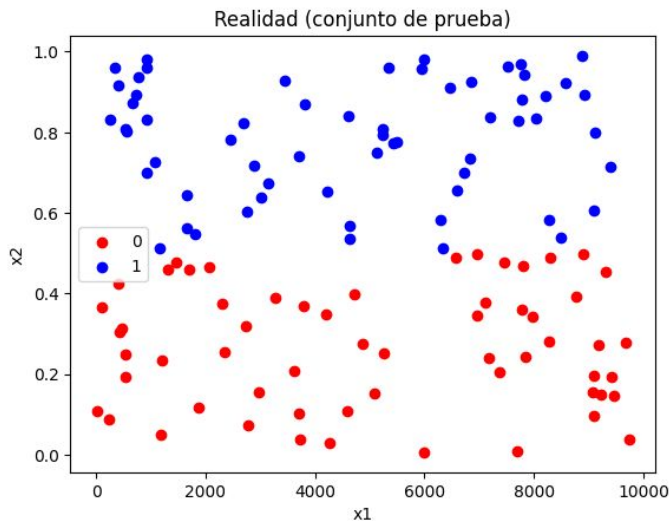
# Entrenamos el modelo usando los datos de entrenamiento
model.fit(X_train, y_train)

# Realizamos la predicción usando los datos de prueba
prediction = model.predict(X_test)

# Generamos la matriz de confusión
confusion_matrix(y_test, prediction)
```

Análisis del desempeño

		Predicción	
		close	far
Realidad	close	59	0
	far	61	0



Escalamiento de las variables

```
import numpy as np
import pandas as pd
from sklearn import neighbors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix

# Creamos un escalador de datos
scaler = StandardScaler()

# Se ajusta el escalador usando únicamente los datos de entrenamiento
scaler.fit(X_train)

# Escalamos las variables independientes del entrenamiento y prueba
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

Se genera el modelo usando los datos escalados

```
# Creamos un modelo de clasificación usando SVM (con kernel RBF)
model = svm.SVC()

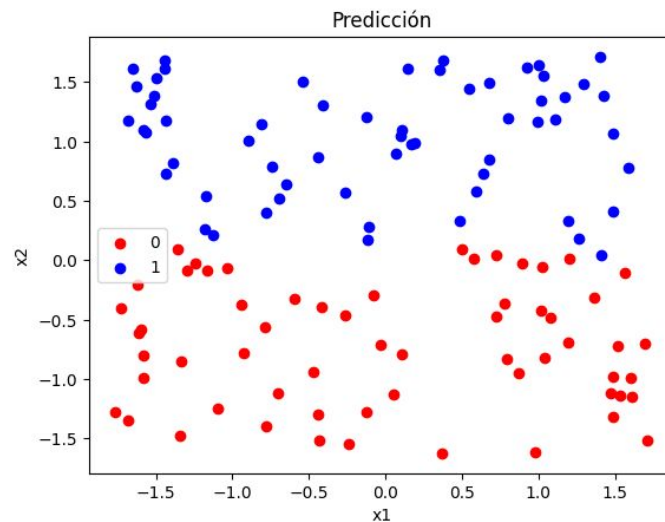
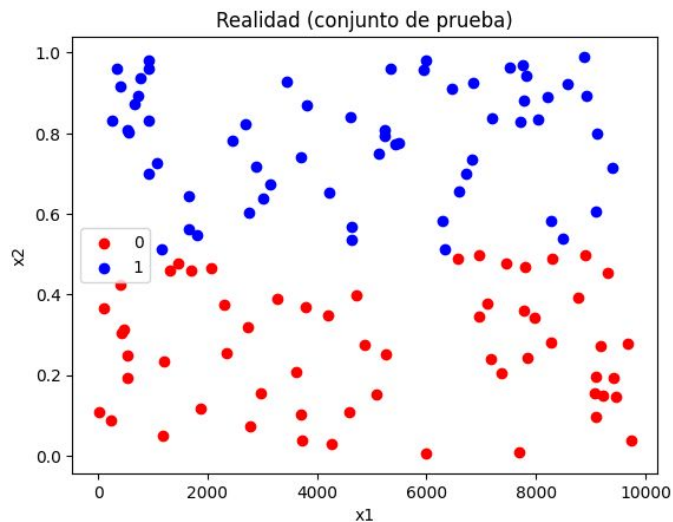
# Entrenamos el modelo usando los datos de entrenamiento
model.fit(X_train, y_train)

# Realizamos la predicción usando los datos de prueba
prediction = model.predict(X_test)

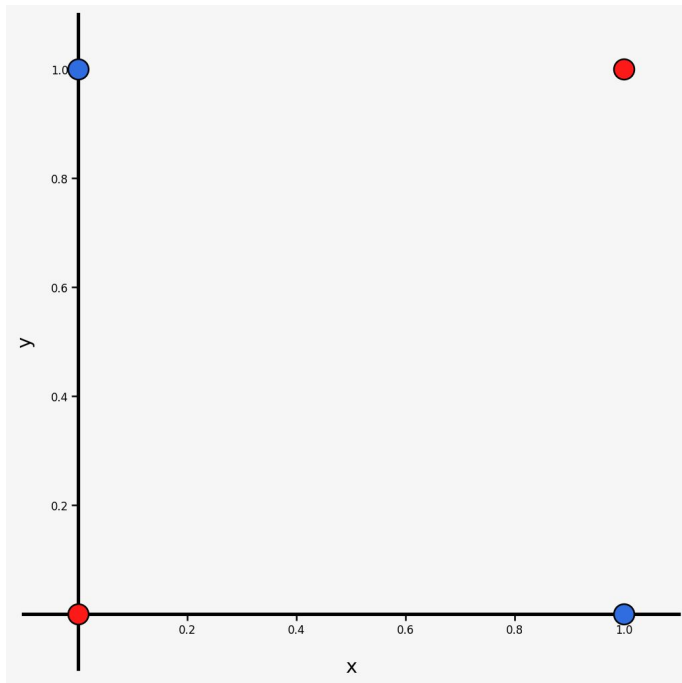
# Generamos la matriz de confusión
confusion_matrix(y_test, prediction)
```

Análisis del desempeño

		Predicción	
		close	far
Realidad	close	58	1
	far	2	59



Entendiendo el funcionamiento del kernel



Punto	x	y	Class
A	0	0	-1
B	0	1	1
C	1	0	1
D	1	1	-1

Conclusiones

- La idea que soporta las SVMs también puede ser usada para regresión mediante regresores de vectores de soporte (SVR).
- Normalmente, las SVMs son resistentes al ruido y al sobreajuste (siempre que sus parámetros sean adecuados).
- La interpretación de las decisiones del modelo suele ser compleja (en algunos aspectos casi es un método de caja negra).
- El entrenamiento de la SVM suele ser costoso. Son recomendables en casos de datasets pequeños o medianos, aún con muchas variables (si las distancias siguen siendo discriminatorias).
- El proceso requiere escalar los datos que el aprendizaje funcione adecuadamente.